ReconX - Complete Project Overview

📁 Project Structure

```
reconx/
├── reconx.py                 # Main orchestrator (Python 3.11+)
├── query.py                  # Database query tool
├── setup.sh                  # Quick setup script
├── install.sh                # Full installation script
├── config.yaml               # Configuration file
├── requirements.txt          # Python dependencies
├── README.md                 # Main documentation
├── LICENSE                   # MIT License
├── .env.example              # API keys template
├── .gitignore                # Git ignore rules
│
├── db/
│   ├── __init__.py
│   └── database.py           # SQLite database manager (Singleton, WAL
mode)
│
├── modules/                  # Bash execution modules
│   ├── 01_discovery.sh       # Phase 1: Discovery & Enumeration
│   ├── 02_intel.sh           # Phase 2: Intelligence & Infrastructure
│   ├── 03_content.sh         # Phase 3: Deep Web & Content
│   └── 04_vuln.sh            # Phase 4: Vulnerability Scanning
│
├── parsers/
│   ├── __init__.py
│   └── parser.py             # Tool output parsers
│
├── examples/
│   └── quick_start.sh        # Usage examples
│
├── output/                   # Generated during scans
│   └── [target]/
│       ├── phase1_discovery/
│       ├── phase2_intel/
│       ├── phase3_content/
│       └── phase4_vulnscan/
│
└── logs/                     # Generated during execution
    └── reconx.log
```

🎯 Core Components

1. Main Orchestrator (reconx.py)

Purpose: Central control system that manages the entire reconnaissance workflow

Key Features:

- Multi-threaded target scanning
- Phase-based execution control
- Database integration
- Progress tracking and resume capability
- Tool output parsing orchestration

Main Classes:

- `ReconX`: Core orchestrator class
- `Colors`: Terminal output formatting

2. Database Manager (db/database.py)

Purpose: Singleton database manager with thread-safe operations

Key Features:

- SQLite3 with WAL mode for concurrent access
- Thread-local connections
- Bulk insert operations
- Comprehensive schema with indexes

Tables:

- `scan_progress` - Phase completion tracking
- `acquisitions` - Company/domain relationships
- `subdomains` - Discovered subdomains with metadata
- `ports` - Open ports and services
- `urls` - Discovered URLs
- `leaks` - Secrets and sensitive data
- `vulnerabilities` - Security findings
- `infrastructure` - OSINT and infra data

3. Bash Modules (modules/*.sh)

Purpose: Execute and chain reconnaissance tools

Phase 1 (01_discovery.sh):

- Subdomain enumeration (10+ tools)
- DNS resolution
- HTTP validation
- Output: Subdomains, alive hosts

Phase 2 (02_intel.sh):

- Port scanning (RustScan + Nmap)

- OSINT (Shodan, Censys)
- Takeover detection (Subjack)
- Repository scanning (Gitleaks, GitHunt)
- Output: Ports, infrastructure data, leaks

Phase 3 (03_content.sh):

- Archive crawling (gau, waybackurls, etc.)
- Directory brute-forcing (ffuf, Feroxbuster, Dirsearch)
- JavaScript analysis (LinkFinder, SecretFinder)
- Pastebin monitoring
- API discovery
- Output: URLs, paths, JS secrets

Phase 4 (04_vuln.sh):

- Nuclei scanning (all templates)
- (config/image/dependency)
- XSS testing (Dalfox)
- SQL injection (SQLMap)
- CORS misconfiguration (Corsy)
- CMS scanning (WPScan, CMSmap)
- SSL/TLS testing (testssl.sh)
- Output: Vulnerabilities by severity

4. Output Parsers (parsers/parser.py)

Purpose: Convert diverse tool outputs into structured data

Parser Classes:

- `SubdomainParser` - Parse subdomain enumeration tools
- `HttpParser` - Parse HTTPx, SubProber
- `DnsParser` - Parse DNSx
- `PortParser` - Parse Nmap XML, RustScan
- `UrlParser` - Parse gau, waybackurls, etc.
- `DirectoryParser` - Parse ffuf, Feroxbuster, Dirsearch
- `VulnerabilityParser` - Parse Nuclei, Dalfox, SQLMap, Corsy
- `LeakParser` - Parse Gitleaks, SecretFinder, LinkFinder
- `TakeoverParser` - Parse Subjack

5. Query Tool (query.py)

Purpose: Interactive database querying and reporting

Features:

- List all targets
- Target summaries with statistics

- View subdomains, vulnerabilities, leaks, ports
- Filter by severity, alive status
- Export to CSV

🔧 Tool Integration

Complete Tool List (50+ Tools)

Category	Tools
Subdomain Enum	Sublist3r, Amass, Assetfinder, Subfinder, SubDominator, crt.sh, SecurityTrails
DNS	DNSx, Dnsprober, Dnsbruter
HTTP	HTTPx, SubProber
Port Scanning	RustScan, Nmap, Masscan
OSINT	Shodan, ShodanX, Censys, Whois
Crawling	gau, waybackurls, hakrawler, Katana, gospider
Directories	ffuf, Feroxbuster, Dirsearch
JS Analysis	LinkFinder, SecretFinder, JSScanner, Retire.js
API Discovery	Kiterunner, Arjun, Newman
Vuln Scanning	Nuclei, Nikto, Wapiti
XSS	Dalfox, XSStrike
SQLi	SQLMap
CORS	Corsy
Takeover	Subjack, SubOver
Leaks	Gitleaks, TruffleHog, GitHunt, git-secrets
CMS	WPScan, CMSmap
SSL/TLS	testssl.sh, SSLyze
Pastebin	PasteHunter, Pastebin API

📊 Data Flow

```
1. User Input (Target Domain)
   ↓
2. ReconX Orchestrator
   ↓
3. Phase Execution (Bash Modules)
   ↓
4. Tool Execution (50+ Tools)
```

```
    ↓
5. Raw Output (JSON/Text/XML)
    ↓
6. Parsers (Python)
    ↓
7. Database Storage (SQLite)
    ↓
8. Query Tool / Reports
```

🗄️ Database Schema Details

scan_progress

Tracks completion of each phase per target

```sql
CREATE TABLE scan_progress (
    target TEXT PRIMARY KEY,
    phase1_done BOOLEAN,
    phase2_done BOOLEAN,
    phase3_done BOOLEAN,
    phase4_done BOOLEAN,
    started_at TIMESTAMP,
    updated_at TIMESTAMP
);
```

subdomains

Comprehensive subdomain data with source tracking

```sql
CREATE TABLE subdomains (
    id INTEGER PRIMARY KEY,
    target TEXT,
    host TEXT,
    ip TEXT,
    is_alive BOOLEAN,
    status_code INTEGER,
    source_tools TEXT,  -- Comma-separated list
    discovered_at TIMESTAMP,
    UNIQUE(target, host)
);
```

vulnerabilities

All security findings with severity

```sql
CREATE TABLE vulnerabilities (
    id INTEGER PRIMARY KEY,
    target TEXT,
    host TEXT,
    tool TEXT,
    severity TEXT,
    name TEXT,
    info TEXT,
    cve TEXT,
    discovered_at TIMESTAMP
);
```

🚀 Execution Flow

Phase 1: Discovery

1. Initialize target in database
2. Run horizontal discovery (Amass intel, Whois)
3. Launch parallel subdomain enumeration
4. Merge and deduplicate results
5. Resolve DNS (DNSx)
6. Validate HTTP (HTTPx)
7. Parse results into database
8. Mark phase complete

Phase 2: Intelligence

1. Validate alive hosts (SubProber)
2. Fast port scan (RustScan)
3. Deep port scan (Nmap)
4. OSINT gathering (Shodan, Censys)
5. Takeover detection (Subjack)
6. Repository scanning (Gitleaks)
7. Parse results into database
8. Mark phase complete

Phase 3: Content Discovery

1. Archive crawling (parallel execution)
2. Merge discovered URLs
3. Directory brute-forcing (top N hosts)
4. JavaScript analysis
5. Pastebin monitoring
6. API discovery
7. Parse results into database
8. Mark phase complete

Phase 4: Vulnerability Scanning

1. Nuclei comprehensive scan
2. config/image scanning
3. XSS testing (Dalfox)
4. SQL injection testing (SQLMap)
5. CORS testing (Corsy)
6. CMS-specific scans
7. SSL/TLS testing
8. Parse results into database
9. Mark phase complete

⚙️ Configuration Options

Via config.yaml

- Tool selection per phase
- Rate limits and timeouts
- Thread counts
- Wordlist paths
- Output formats
- Severity filters
- Exclusion patterns

Via Environment Variables

- API keys (Shodan, Censys, GitHub, etc.)
- Custom user agents
- Proxy settings

Via Command Line

- Target selection
- Phase selection
- Output directory
- Thread count
- Database path

🔍 Query Examples

CLI Queries

```
# List all targets
python3 query.py --list

# Show target summary
python3 query.py -t example.com --summary
```

```bash
# View alive subdomains
python3 query.py -t example.com --subdomains --alive-only

# View critical vulnerabilities
python3 query.py -t example.com --vulns --severity critical

# Export to CSV
python3 query.py -t example.com --export vulnerabilities -o vulns.csv
```

Direct SQL Queries

```sql
-- Get all critical findings
SELECT tool, name, host, cve
FROM vulnerabilities
WHERE target = 'example.com' AND severity = 'critical';

-- Subdomain statistics by source
SELECT source_tools, COUNT(*) as count
FROM subdomains
WHERE target = 'example.com'
GROUP BY source_tools;

-- Top ports found
SELECT port, service, COUNT(*) as count
FROM ports
WHERE target = 'example.com'
GROUP BY port, service
ORDER BY count DESC;
```

🛡️ Error Handling

Bash Modules

- Each tool execution wrapped in error handling
- Tools failures logged but don't stop phase
- Parallel execution with wait for completion
- Timeout protection per tool

Python Orchestrator

- Try-catch blocks around module execution
- Database transaction safety
- Thread-safe operations
- Resume capability on failure

📈 Performance Characteristics

Concurrency

- Multi-target scanning via ThreadPoolExecutor
- Parallel tool execution within phases
- Thread-safe database operations (WAL mode)

Resource Usage

- Memory: ~2-4GB typical
- Disk: Varies by target size (1-10GB+)
- CPU: Multi-threaded tool execution
- Network: Rate-limited per tool

Timing (Typical)

- Phase 1: 10-30 minutes
- Phase 2: 30-60 minutes
- Phase 3: 45-90 minutes
- Phase 4: 60-120 minutes
- **Total: 2-5 hours** per target

🔐 Security Considerations

Operational Security

- Use VPN/proxy for scanning
- Respect rate limits
- API key security (.env file)
- Log sanitization

Tool Safety

- All tools run as subprocess
- Timeout protection
- No arbitrary code execution
- Input validation

Data Privacy

- Local database storage
- No external data transmission (except tool API calls)
- Sensitive data in .env file (gitignored)

📦 Deployment

Development

```
git clone <repo>
cd reconx
bash setup.sh
```

```
source .env
python3 reconx.py -t example.com
```

Production

```
# Full installation
sudo bash install.sh

# Configure API keys
cp .env.example .env
vim .env

# Run with logging
python3 reconx.py -t example.com 2>&1 | tee scan.log
```

Docker (Future)

```
FROM kalilinux/kali-rolling
# Install dependencies
# Copy ReconX files
# ENTRYPOINT ["python3", "reconx.py"]
```

🔄 Future Enhancements

Planned Features

- ☐ HTML/PDF report generation
- ☐ Web UI dashboard
- ☐ Real-time notifications (Slack, Discord)
- ☐ Distributed scanning
- ☐ Cloud integration (AWS, GCP, Azure)
- ☐ Machine learning for false positive reduction
- ☐ CI/CD integration
- ☐ Docker containerization
- ☐ Kubernetes deployment
- ☐ REST API

Tool Additions

- Additional CMS scanners
- Cloud misconfiguration scanners
- API security testing tools
- Mobile app analysis
- IoT device scanning

📚 Resources

Documentation

- README.md - Main documentation
- config.yaml - Configuration reference
- examples/ - Usage examples

Support

- GitHub Issues
- Wiki (planned)
- Discord (planned)

🎓 Learning Path

For Users

1. Read README.md
2. Run setup.sh
3. Try examples/quick_start.sh
4. Run single-phase scans
5. Run full scans
6. Query results with query.py

For Developers

1. Study architecture (this file)
2. Understand database schema
3. Review parser implementations
4. Add new tools to modules
5. Create new parsers
6. Submit pull requests

ReconX Version 1.0
Last Updated: 2024
License: MIT